

Konzept und Reflexion

Ein Uptime-Monitor mit Python, Prometheus und Grafana

Ibrade Rhicko · github.com/ibradericko/uptime-monitor · Stand September 2026

INHALT

- 1 Worum es geht
- 2 Aufbau
- 3 Was gemessen wird
- 4 Drumherum
- 5 Reflexion
- 6 Begriffe

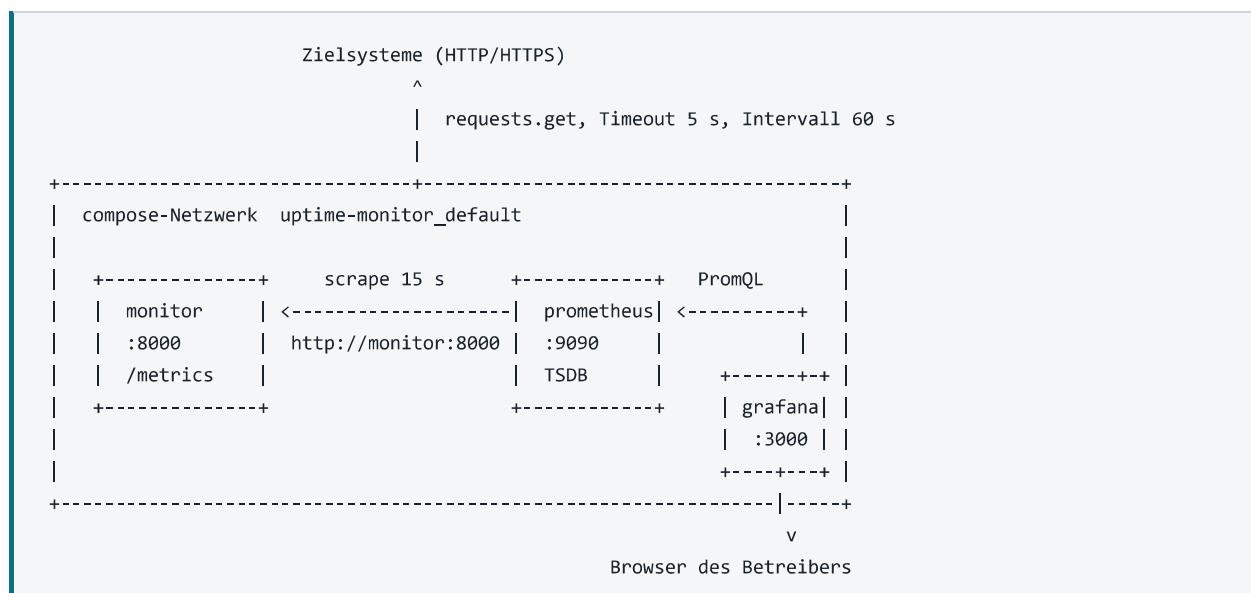
1 Worum es geht

Eine Website fällt nachts aus, ein Kunde merkt es am nächsten Morgen als Erster und ruft an. Genau diesen Moment will ein Uptime-Monitor verhindern: ein Programm, das regelmässig selbst nachfragt, ob eine Seite noch antwortet, und das Ergebnis festhält.

Solche Werkzeuge gibt es fertig zu kaufen. Ich wollte trotzdem einmal selbst durch die ganze Kette gehen, vom Skript über Tests und Systemdienst bis zum Dashboard, das am Ende den Verlauf zeigt.

2 Aufbau

Drei Programme laufen zusammen, jedes in seinem eigenen Container:



Mein Skript ruft die Websites auf und schreibt das Ergebnis unter `/metrics`. Alle 15 Sekunden holt sich Prometheus diese Werte und speichert sie ab. Grafana nimmt sich das und macht daraus das Dashboard.

Die drei finden sich über ihren Namen im Netzwerk: Prometheus kennt meinen Monitor einfach als `monitor:8000`, ganz ohne IP-Adresse. Konfiguration liegt komplett als Datei im Repository. Nur die eigentlichen Messwerte liegen in einem eigenen Speicherbereich, der einen Neustart übersteht.

3 Was gemessen wird

Für jede URL messe ich drei Dinge: ob sie antwortet, mit welchem Statuscode, und wie lange die Antwort gedauert hat.

Metrik	Bedeutung
uptime_erreichbar	1 = Antwort erhalten, 0 = keine
uptime_status	HTTP-Statuscode der Antwort
uptime_versuchsdauer_sekunden	Dauer des Versuchs, auch im Fehlerfall

Erreichbarkeit und Statuscode beantworten unterschiedliche Fragen. Eine Seite mit einem 404 ist erreichbar, der Server lebt ja, nur die angefragte Seite gibt es nicht. Das ist ein Problem der Anwendung, kein Netzwerkproblem, und dafür ruft man im Ernstfall jemand anderen an.

Ausserdem unterscheide ich, warum eine Antwort ausbleibt: kein Name gefunden, keine Verbindung, oder eine Verbindung, die einfach nie antwortet. Drei Ursachen, die in der Praxis auch drei verschiedene Leute betreffen.

4 Drumherum

Rundherum ist einiges entstanden, das mit dem eigentlichen Skript nur noch am Rand zu tun hat:

Bereich	Umsetzung
Anwendung	Python 3.11, requests, prometheus_client
Tests	pytest, fünf Tests, ohne echten Netzwerkzugriff
Betrieb	Docker Compose, alternativ systemd
Pipeline	GitHub Actions, Tests und Image-Build bei jedem Push
Erfassung & Darstellung	Prometheus und Grafana, als Code provisioniert

Die Tests laufen ohne echtes Netzwerk, sie ersetzen die Antwort von `requests.get` durch ein Testobjekt. Die Pipeline prüft nicht, ob es bei mir läuft, das weiss ich ja schon, sondern ob es auch auf einer fremden Maschine läuft, die nichts über mich weiss.

5 Reflexion

Ich komme aus dem Microsoft-Umfeld, mehrere Jahre als System-Administrator: viel Windows Server, viel Microsoft 365, kaum Linux. DevOps, Container, Infrastructure as Code waren Begriffe, die ich kannte, aber nie wirklich angefasst hatte. Dieses Projekt war der erste Ort, an dem ich das nachgeholt habe.

Am meisten gelernt habe ich nicht in einem Erfolgsmoment, sondern bei einem Fehler, der lange unentdeckt blieb. Fiel eine Website aus, blieb ihr letzter Statuscode im Dashboard einfach stehen: eine 200, obwohl seit Stunden niemand mehr antwortete. Die Tests waren grün, das Dashboard sah unauffällig aus. Aufgefallen ist es mir erst, als ich mich selbst gefragt habe, was eigentlich passieren müsste, wenn eine Seite ausfällt, und das mit der Realität verglichen habe. Seitdem prüfe ich nicht mehr nur, ob etwas läuft, sondern auch, ob es sich so verhält, wie ich es erwarte.

Denselben Punkt habe ich später noch einmal erlebt, als ich das Projekt auf einem zweiten, fremden Rechner installiert habe. Bei mir lief alles, dort blieb das Dashboard leer, ohne jede Fehlermeldung. Der Grund waren Dateirechte, die Git gar nicht mitüberträgt und die auf dem zweiten System einfach enger waren. Eine Pipeline hätte das nie gefunden, sie baut nur das Image, den Stack startet sie nie wirklich. Erst der Test auf einer Maschine, die ich nicht selbst eingerichtet hatte, hat es gezeigt.

Am meisten Spass hat mir die Arbeit mit Linux gemacht. Vermutlich, weil dort meine Lücke am grössten war. Wie systemd einen Dienst überwacht, wie ein Prozess sich verhält, warum ein belegter Port eine Fehlermeldung erzeugt, die aussieht wie ein Konfigurationsfehler: Vieles davon habe ich erst verstanden, als es einmal nicht funktioniert hat.

Was sich für mich am stärksten verändert hat, ist die Art, wie ich über Änderungen denke. Bisher habe ich Einstellungen im Admin Center geklickt oder per PowerShell gesetzt. Beides funktioniert, aber danach ist nur das Ergebnis da, nicht der Weg dorthin. In diesem Projekt steht der gewünschte Zustand in einer Datei, die ich versionieren, nachlesen und wiederholen kann. Das ist der Teil, der mir am meisten bringt, wenn ich an die nächste Stelle denke.

Was noch fehlt, weiss ich ziemlich genau: Die Metrik-Logik ist noch nicht in eine eigene, testbare Funktion ausgelagert, eine echte Alarmregel gibt es noch nicht, und PromQL kann ich lesen, aber noch nicht aus dem Stand schreiben. Das Projekt ist ein Anfang, kein Abschluss. Aber einer, an dem ich zum ersten Mal wirklich verstanden habe, wovon ich vorher nur die Begriffe kannte.

6 Begriffe

Begriff	Bedeutung
Blackbox-Monitoring	Prüfung von aussen, ohne Kenntnis der inneren Vorgänge — so wie ein Benutzer den Dienst erlebt.
Statuscode	Zahl, die ein Webserver mit jeder Antwort mitschickt. 200 in Ordnung, 404 nicht gefunden, 500 Fehler auf dem Server.
DNS	Namensdienst des Internets. Übersetzt Namen wie example.com in die Adresse, unter der der Rechner erreichbar ist.
Metrik	Ein Messwert als Zahl, den eine Auswertungssoftware regelmässig abholt.
Gauge	Metriktyp für Werte, die steigen und fallen können — im Gegensatz zum Counter, der nur wächst.
Zeitreihe	Eine Folge von Wertepaaren aus Zeitstempel und Zahl.
Scrape	Der regelmässige Abruf des Metrik-Endpunkts durch Prometheus.
Container	Paket aus Anwendung und vollständiger Laufzeitumgebung.
Provisionierung	Einrichtung aus Konfigurationsdateien beim Start statt durch Klicks in der Oberfläche.
Pipeline	Automatische Prüfstrecke, die bei jeder Änderung Tests ausführt und das Paket baut.
Repository	Versionierte Ablage aller Dateien mit vollständiger Änderungsgeschichte.